

10 | 再探trait：带类型参数的trait及trait object

唐刚 · Rust 语言从入门到实战



你好，我是 Mike，今天我们继续学习 trait 相关知识。

回顾一下我们上一节课中类型参数出现的地方。

用 trait 对 T 作类型空间的约束，比如 `T: TraitA`。

blanket implementation 时的 T，比如 `impl<T: TraitB> TraitA for T {}`。

函数里的 T 参数，比如 `fn doit<T>(a: T) {}`。

你要注意区分不同位置的 T。它的基础意义都是类型参数，但是放在不同的位置其侧重的意义有所不同。

`T: TraitA` 里的 T 表示类型参数，强调“参数”，使用 TraitA 来削减它的类型空间。

`impl<T: TraitB> TraitA for T {}` 末尾的 T 更强调类型参数的“类型”部分，为某些类型实现 TraitA。

`doit<T>(a: T) {}` 中第二个 `T` 表示某种类型，更强调类型参数的“类型”部分。

这节课我们要讲的是另外一个东西，它里面也带 `T` 参数。我们一起来看一下，它与之前这几种形式有什么不同。

trait 上带类型参数

trait 上也是可以带类型参数的，形式像下面这样：

```
1 trait TraitA<T> {}
```

[📄 复制代码](#)

表示这个 trait 里面的函数或方法，可能会用到这个类型参数。在定义 trait 的时候，还没确定这个类型参数的具体类型。要等到 `impl` 甚至使用类型方法的时候，才会具体化这个 `T` 的具体类型。

注意，这个时候 `TraitA<T>` 是一个整体，表示一个 trait。比如 `TraitA<u8>` 和 `TraitA<u32>` 就是两个不同的 trait，这里单独把 `TraitA` 拿出来说是没有意义的。

实现时需要在 `impl` 后面先定义类型参数，比如：

```
1 impl<T> TraitA<T> for Atype {}
```

[📄 复制代码](#)

当然也可以在对类型实现时，将 `T` 参数具体化，比如：

```
1 impl TraitA<u8> for Atype {}
```

[📄 复制代码](#)

而如果被实现的类型上自身也带类型参数，那么情况会更复杂。

 复制代码

```
1 trait TraitA<T> {}
2 struct Atype<U> {
3     a: U,
4 }
5 impl<T, U> TraitA<T> for Atype<U> {}
```

这些类型参数都是可以在 impl 时被约束的，像下面这样：

 复制代码

```
1 use std::fmt::Debug;
2
3 trait TraitA<T> {}
4 struct Atype<U> {
5     a: U,
6 }
7 impl<T, U> TraitA<T> for Atype<U>
8 where
9     T: Debug,          // 在 impl 时添加了约束
10    U: PartialEq,      // 在 impl 时添加了约束
11 {}
```

注：以上代码都是可以放到 playground 中编译通过的。

impl 示例

下面我们通过一个具体的实例体会一下带类型参数的 trait 的威力。

我们现在要实现一个模型。

1. 平面上的一个点与平面上的另一个点相加，形成一个新的点。算法是两个点的 x 分量和 y 分量分别相加。
2. 平面上的一个点加一个整数 i32，形成一个新的点。算法是分别在 x 分量和 y 分量上面加这个 i32 参数。

本示例借鉴了： <https://github.com/pretzelhammer/rust-blog/blob/master/posts/tour-of-rusts-standard-library-traits.md#generic-types-vs-associated-types>

代码如下：

 复制代码

```
1 // 定义一个带类型参数的trait
2 trait Add<T> {
3     type Output;
4     fn add(self, rhs: T) -> Self::Output;
5 }
6
7 struct Point {
8     x: i32,
9     y: i32,
10 }
11
12 // 为 Point 实现 Add<Point> 这个 trait
13 impl Add<Point> for Point {
14     type Output = Self;
15     fn add(self, rhs: Point) -> Self::Output {
16         Point {
17             x: self.x + rhs.x,
18             y: self.y + rhs.y,
19         }
20     }
21 }
22
23 // 为 Point 实现 Add<i32> 这个 trait
24 impl Add<i32> for Point {
25     type Output = Self;
26     fn add(self, rhs: i32) -> Self::Output {
27         Point {
28             x: self.x + rhs,
29             y: self.y + rhs,
30         }
31     }
32 }
33
34 fn main() {
35     let p1 = Point { x: 1, y: 1 };
36     let p2 = Point { x: 2, y: 2 };
37     let p3 = p1.add(p2); // 两个Point实例相加
38     assert_eq!(p3.x, 3);
```



```
39     assert_eq!(p3.y, 3);
40
41     let p1 = Point { x: 1, y: 1 };
42     let delta = 2;
43     let p3 = p1.add(delta);    // 一个Point实例加一个i32
44     assert_eq!(p3.x, 3);
45     assert_eq!(p3.y, 3);
46 }
```

我们详细解释一下这个示例。Add<T> 这个 trait，带一个类型参数 T，还带一个关联类型 Output。

对 Point 类型，我们实现了两个 trait：Add<Point> 和 Add<i32>。注意这已经是两个不同的 trait 了，所以能对同一个类型实现。前面我们反复强调过，同一个 trait 只能对一个类型实现一次。

根据需求，运算后的类型也是 Point，所以看到两个 trait 中的关联类型都是 Self。请注意两个 trait 中实现的不同算法。

通过这种形式，我们在同一个类型上实现了同名方法（add 方法）参数类型的多种形态。在这里看起来就是，Point 实例的 add 方法既可以接收 Point 参数，又可以接收 i32 参数，Rustc 小助手可以根据不同的参数类型自动找到对应的方法调用。在 Java、C++ 这些语言中，有语言层面的函数重载特性来支持这种功能，Rust 中自身并不直接支持函数重载特性，但是它用 trait 就轻松实现了同样的效果，这是一种全新的思路。

trait 类型参数的默认实现

定义带类型参数的 trait 的时候，可以为类型参数指定一个默认类型，比如 trait TraitA<T = u64> {}。这样使用时，impl TraitA for SomeType {} 就等价于 impl TraitA<u64> for SomeType {}。

我们来看一个完整的例子。

 复制代码

```
1 // Self可以用在默认类型位置上
```

```

2 trait TraitA<T = Self> {
3     fn func(t: T) {}
4 }
5
6 // 这个默认类型为i32
7 trait TraitB<T = i32> {
8     fn func2(t: T) {}
9 }
10
11 struct SomeType;
12
13 // 这里省略了类型参数，所以这里的T为Self
14 // 进而T就是SomeType本身
15 impl TraitA for SomeType {
16     fn func(t: SomeType) {}
17 }
18 // 这里省略了类型参数，使用默认类型i32
19 impl TraitB for SomeType {
20     fn func2(t: i32) {}
21 }
22 // 这里不省略类型参数，明确指定类型参数为String
23 impl TraitA<String> for SomeType {
24     fn func(t: String) {}
25 }
26 // 这里不省略类型参数，明确指定类型参数为String
27 impl TraitB<String> for SomeType {
28     fn func2(t: String) {}
29 }

```

默认参数给表达上带来了一定程度的简洁，但是增加了初学者识别和理解上的困难。

你还记得上一节课讲关联类型时我们提到过在使用约束时可以具化关联类型。那里也是用的 = 号。比如：

 复制代码

```

1 trait TraitA {
2     type Item;
3 }
4 // 这里，定义结构体类型时，用到了TraitA作为约束
5 struct Foo<T: TraitA<Item=String>> {
6     x: T
7 }

```

初看这里容易混淆。区别在于，**关联类型的具化是在应用约束时，类型参数的默认类型指定是在定义 trait 时**，通过 trait 出现的场景可以区分它们。

trait 中的类型参数与关联类型的区别

现在你可能会有些疑惑：trait 上的类型参数和关联类型都具有延迟具化的特点，那么它们的区别是什么呢？为什么要设计两种不同的机制呢？

首先要明确的一点是，Rust 本身也在持续演化过程中。有些特性先出现，有些特性是后出现的。最后演化出功能相似但是不完全一样的特性是完全有可能的。

具体到这两者来说，它们主要有两点不同。

1. 类型参数可以在 impl 类型的时候具化，也可以延迟到使用的时候具化。而关联类型在被 impl 时就必须具化。
2. 由于类型参数和 trait 名一起组成了完整的 trait 名字，不同的具化类型会构成不同的 trait，所以看起来同一个定义可以在目标类型上实现“多次”。而关联类型没有这个作用。

下面我们分别举例说明。

对于第一点，请看下面的示例：

 复制代码

```
1 use std::fmt::Debug;
2
3 trait TraitA<T>
4 where
5     T: Debug, // 定义TraitA<T>的时候，对T作了约束
6 {
7     fn play(&self, _t: T) {}
8 }
9 struct Atype;
10
11 impl<T> TraitA<T> for Atype
12 where
13     T: Debug + PartialEq, // 将TraitA<T>实现到类型Atype上时，加强了约束
14 {}
```

```
15 fn main() {  
16     let a = Atype;  
17     a.play(10u32); // 在使用时，通过实例方法传入的参数类型具化T  
18 }  
19
```

这个示例展示了几个要点。

1. 定义带类型参数的 trait 时可以用 where 表达，并提供约束。
2. impl trait 时可以对类型参数加强约束，对应例子中的 Debug + PartialEq。
3. impl trait 时可以不具化类型参数。
4. 可以在使用方法时具化类型参数。例子中的 `a.play(10u32)`，把 T 具象化成了 u32。

当然，在 impl 的时候也可以指定成 u32 类型，所以下面的代码也可以。

[复制代码](#)

```
1 use std::fmt::Debug;  
2  
3 trait TraitA<T>  
4 where  
5     T: Debug,  
6 {  
7     fn play(&self, _t: T) {}  
8 }  
9 struct Atype;  
10  
11 impl TraitA<u32> for Atype {} // 这里具化成了 TraitA<u32>  
12  
13 fn main() {  
14     let a = Atype;  
15     a.play(10u32);  
16 }
```

但是这样就没前面那么灵活了，比如 `a.play(10u64)` 就不行了。

对应的，对关联类型来说，如果你在 impl 时不对其具化，就无法编译通过。所以对于第二点，我也给出一个例子来解释。我们把前面对 Point 类型实现 Add 的模型尝试用关联类型实

现一遍。

[📄 复制代码](#)

```
1 trait Add {
2     type ToAdd;    // 多定义一个关联类型
3     type Output;
4     fn add(self, rhs: Self::ToAdd) -> Self::Output;
5 }
6
7 struct Point {
8     x: i32,
9     y: i32,
10 }
11
12 impl Add for Point {
13     type ToAdd = Point;
14     type Output = Point;
15     fn add(self, rhs: Point) -> Point {
16         Point {
17             x: self.x + rhs.x,
18             y: self.y + rhs.y,
19         }
20     }
21 }
22
23 impl Add for Point { // 这里重复impl了同一个trait, 无法编译通过
24     type ToAdd = i32;
25     type Output = Point;
26     fn add(self, rhs: i32) -> Point {
27         Point {
28             x: self.x + rhs,
29             y: self.y + rhs,
30         }
31     }
32 }
33
34 fn main() {
35     let p1 = Point { x: 1, y: 1 };
36     let p2 = Point { x: 2, y: 2 };
37     let p3 = p1.add(p2);
38     assert_eq!(p3.x, 3);
39     assert_eq!(p3.y, 3);
40
41     let p1 = Point { x: 1, y: 1 };
42     let delta = 2;
43     let p3 = p1.add(delta); // 这句是错的
44     assert_eq!(p3.x, 3);
```

```
45     assert_eq!(p3.y, 3);
```

编译器会抱怨：

[复制代码](#)

```
1 error[E0119]: conflicting implementations of trait `Add` for type `Point`:
2   --> src/main.rs:23:1
3   |
4 12 | impl Add for Point {
5   | ----- first implementation here
6   ...
7 23 | impl Add for Point {
8   | ^^^^^^^^^^^^^^^^^^^ conflicting implementation for `Point`
```

提示说，对 Point 类型实现了多次 Add，导致冲突。编译不通过。所以这个模型仅用关联类型来实现，是写不出来的。

这么看起来，好像带类型参数的 trait 功能更强大，那用这个不就够了？但关联类型也有它的优点，比如关联类型没有类型参数，不存在多引入了一个参数的问题，而类型参数是具有传染性的，特别是在一个调用层次很深的系统中，增删一个类型参数可能会导致整个项目文件到处都需要改，非常头疼。

而关联类型没有这个问题。在一些场合下，关联类型正好是减少类型参数数量的一种方法。更不要说，有时模型比较简单，不需要多态特性，这时用关联类型就更简洁，代码可读性更好。

trait object

下面我们开始讲 trait object。

我们从一个函数要返回不同的类型说起。比如一个常见的需求，要在一个 Rust 函数中返回可能的多种类型，应该怎么写？

如果我们写成返回固定类型的函数签名，那么它就只能返回那个类型。比如：

 复制代码

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4 fn doit() -> Atype {
5     let a = Atype;
6     a
7 }
```

你想到的第一个办法可能是利用 enum。

 复制代码

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 enum TotalType {
6     A(Atype),    // 用变体把目标类型包起来
7     B(Btype),
8     C(Ctype),
9 }
10
11 fn doit(i: u32) -> TotalType { // 返回枚举类型
12     if i == 0 {
13         let a = Atype;
14         TotalType::A(a)    // 在这个分支中返回变体A
15     } else if i == 1 {
16         let b = Btype;
17         TotalType::B(b)    // 在这个分支中返回变体B
18     } else {
19         let c = Ctype;
20         TotalType::C(c)    // 在这个分支中返回变体C
21     }
22 }
```

enum 常用于聚合类型。这些类型之间可以没有任何关系，用 enum 可以**无脑 + 强行**把它们揉在一起。enum 聚合类型是编码时已知的类型，也就是说在聚合前，需要知道待聚合类型的边界，一旦定义完成，之后运行时就不能改动了，它是**封闭类型集**。

第二种办法是利用类型参数，我们试着引入一个类型参数，改写一下。

 复制代码

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4 fn doit<T>() -> T {
5     let a = Atype;
6     a
7 }
```

很明显，这种代码无法通过编译。提示：

 复制代码

```
1 error[E0308]: mismatched types
2   --> src/lib.rs:6:3
3   |
4 4 | fn doit<T>() -> T {
5   |             -      - expected `T` because of return type
6   |             |
7   |             this type parameter
8 5 |     let a = Atype;
9 6 |     a
10  |     ^ expected type parameter `T`, found `Atype`
11  |
12 = note: expected type parameter `T`
13           found struct `Atype`
```

因为这里这个类型参数 `T` 是在这个函数调用时指定，而不是在这个函数定义时指定的。所以针对我们的需求，你没法在这里直接返回一个具体的类型代入 `T`。只能尝试用 `T` 来返回，于是我们改出第二个版本。

 复制代码

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 impl Atype {
6     fn new() -> Atype {
7         Atype
8     }
9 }
```



```

10 impl Btype {
11     fn new() -> Btype {
12         Btype
13     }
14 }
15
16 impl Ctype {
17     fn new() -> Ctype {
18         Ctype
19     }
20 }
21
22 fn doit<T>() -> T {
23     T::new()
24 }
25

```

编译还是报错。

 复制代码

```

1 error[E0599]: no function or associated item named `new` found for type parameter
2   --> src/main.rs:24:6
3   |
4 23 | fn doit<T>() -> T {
5   |             - function or associated item `new` not found for this type paramete
6 24 |     T::new()
7   |         ^^^ function or associated item not found in `T`
8

```

也就是说，Rustc 小助手并不知道我们定义这个类型参数 T 里面有 new 这个关联函数。联想到我们前面学过的，可以用 trait 来定义这个协议，让 Rust 认识它。

第三个版本：

 复制代码

```

1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {
6     fn new() -> Self;    // TraitA中定义了new()函数

```

```
7 }
8
9 impl TraitA for Atype {
10     fn new() -> Atype {
11         Atype
12     }
13 }
14
15 impl TraitA for Btype {
16     fn new() -> Btype {
17         Btype
18     }
19 }
20
21 impl TraitA for Ctype {
22     fn new() -> Ctype {
23         Ctype
24     }
25 }
26
27 fn doit<T: TraitA>() -> T {
28     T::new()
29 }
30
31 fn main() {
32     let a: Atype = doit::<Atype>();
33     let b: Btype = doit::<Btype>();
34     let c: Ctype = doit::<Ctype>();
35 }
```

这个版本顺利通过编译。在这个示例中，我们认识到了引入 trait 的必要性，就是让 Rustc 小助手知道我们在协议层面有一个 new() 函数，一旦类型参数被 trait 约束后，它就可以去 trait 中寻找协议定义的函数和方法。

为了解决上面那个问题，我们真的是费了不少力气。实际上，Rust 提供了更优雅的方案来解决这个需求。Rust 利用 trait 提供了一种特殊语法 impl trait，你可以看一下示例。

[复制代码](#)

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
```

```

6  impl TraitA for Atype {}
7  impl TraitA for Btype {}
8  impl TraitA for Ctype {}
9
10 fn doit() -> impl TraitA { // 注意这一行的函数返回类型
11     let a = Atype;
12     a
13     // 或
14     // let b = Btype;
15     // b
16     // 或
17     // let c = Ctype;
18     // c
19 }
20

```

可以看到，这种表达非常简洁，同一个函数签名可以返回多种不同的类型，并且在函数定义时就可以返回具体的类型的实例。更重要的是消除了类型参数 T。

上述代码已经很有用了，但是还是不够灵活，比如我们要用 if 逻辑选择不同的分支返回不同的类型，就会遇到问题。

[复制代码](#)

```

1  struct Atype;
2  struct Btype;
3  struct Ctype;
4
5  trait TraitA {}
6
7  impl TraitA for Atype {}
8  impl TraitA for Btype {}
9  impl TraitA for Ctype {}
10
11 fn doit(i: u32) -> impl TraitA {
12     if i == 0 {
13         let a = Atype;
14         a // 在这个分支中返回类型a
15     } else if i == 1 {
16         let b = Btype;
17         b // 在这个分支中返回类型b
18     } else {
19         let c = Ctype;
20         c // 在这个分支中返回类型c
21     }
22 }

```

22 }

提示：

 复制代码

```

1 error[E0308]: `if` and `else` have incompatible types
2   --> src/lib.rs:22:5
3   |
4 17 |         } else if i == 1 {
5   |         -----
6 18 |         let b = Btype;
7 19 |         b
8   |         - expected because of this
9 20 |     } else {
10 21 |         let c = Ctype;
11 22 |         c
12   |         ^ expected `Btype`, found `Ctype`
13 23 |     }
14   |     |---- `if` and `else` have incompatible types

```

if else 要求返回同一种类型，Rust 检查确实严格。不过我们可以通过加 return 跳过 if else 的限制。

 复制代码

```

1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10
11 fn doit(i: u32) -> impl TraitA {
12     if i == 0 {
13         let a = Atype;
14         return a;           // 这里用return语句直接从函数返回
15     } else if i == 1 {
16         let b = Btype;
17         return b;
18     } else {

```

```

19     let c = CType;
20     return c;
21 }
22 }

```

但是还是报错。

[复制代码](#)

```

1 error[E0308]: mismatched types
2   --> src/lib.rs:19:12
3   |
4 13 | fn doit(i: u32) -> impl TraitA { // 这一行
5   |                               ----- expected `Atype` because of return type
6 ...
7 19 |     return b
8   |           ^ expected `Atype`, found `Btype`
9

```

它说期望 Atype，却得到了 Btype。这个报错其实有点奇怪，它们不是都满足 impl TraitA 吗？

原来问题在于，impl TraitA 作为函数返回值这种语法，其实也只是**指代某一种类型**而已，而这种类型是在函数体中由返回值的类型来自动推导出来的。例子中，Rustc 小助手遇到 Atype 这个分支时，就已经确定了函数返回类型为 Atype，因此当它分析到后面的 Btype 分支时，就发现类型不匹配了。问题就在这里。你可以将条件分支顺序换一下，看一下报错的提示，加深印象。

那我们应该怎么处理这种问题呢？

好在，Rust 还给我们提供了进一步的措施：**trait object**。形式上，就是在 trait 名前加 dyn 关键字修饰，在这个例子里就是 dyn TraitA。**dyn TraitName 本身就是一种类型**，它和 TraitName 这个 trait 相关，但是它们不同，dyn TraitName 是一个独立的类型。

我们使用 dyn TraitA 改写上面的代码。

```

1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10
11 fn doit(i: u32) -> dyn TraitA { // 注意这里的返回类型换成了 dyn TraitA
12     if i == 0 {
13         let a = Atype;
14         return a
15     } else if i == 1 {
16         let b = Btype;
17         return b
18     } else {
19         let c = Ctype;
20         return c
21     }
22 }

```

但是编译会报错。

```

1 error[E0746]: return type cannot have an unboxed trait object
2   --> src/lib.rs:13:20
3   |
4 13 | fn doit(i: u32) -> dyn TraitA {
5   |                               ^^^^^^^^^^^^^ doesn't have a size known at compile-time
6   |
7 help: return an `impl Trait` instead of a `dyn Trait`, if all returned values are
8   |
9 13 | fn doit(i: u32) -> impl TraitA {
10  |                      ~~~~
11 help: box the return type, and wrap all of the returned values in `Box::new`
12  |
13 13 ~ fn doit(i: u32) -> Box<dyn TraitA> {
14 14 |     if i == 0 {
15 15 |         let a = Atype;
16 16 ~         return Box::new(a)
17 17 |     } else if i == 1 {
18 18 |         let b = Btype;

```

```
19 19 ~      return Box::new(b)
20 20 |    } else {
21 21 |      let c = CType;
22 22 ~      return Box::new(c)
```

这段提示很经典，我们来仔细阅读一下。

它说 dyn TraitA 编译时尺寸未知。dyn trait 确实不是一个固定尺寸类型。然后给出了第一个建议：你可以用 impl TraitA 来解决，前提是所有分支返回同一类型。随后给出了第二个建议，你可以用 Box 把 dyn TraitA 包起来。

(🤖：有没有 ChatGPT 的即时感，聪明得不太像一个编译器。)

第一个建议我们已经试过了，Pass，我们按照第二种建议改一下试试。

 复制代码

```
1 struct Atype;
2 struct Btype;
3 struct CType;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for CType {}
10
11 fn doit(i: u32) -> Box<dyn TraitA> {
12     if i == 0 {
13         let a = Atype;
14         Box::new(a)
15     } else if i == 1 {
16         let b = Btype;
17         Box::new(b)
18     } else {
19         let c = CType;
20         Box::new(c)
21     }
22 }
```

这下完美了，编译通过，达成目标，我们成功地将不同类型的实例在同一个函数中返回了。

这里我们引入了一个新的东西 `Box<T>`。`Box<T>` 的作用是可以保证获得里面值的所有权，必要的时候会进行内存的复制，比如把栈上的值复制到堆中去。一旦值到了堆中，就很容易掌握到它的所有权。

具体到这个示例中，因为 `a`、`b`、`c` 都是函数中的局部变量，这里如果返回引用 `&dyn TraitA` 的话是万万不能的，因为违反了所有权规则。而 `Box<T>` 就能满足这里的要求。后续我们在智能指针一讲中会继续讲解 `Box<T>`。

这里我们先暂停，我希望你可以用一点时间来回顾一下整个推导过程，这次令人印象深刻的类型“体操”值得我们多品味几次。

利用 trait object 传参

`impl trait` 和 `dyn trait` 也可以用于函数传参。

`impl trait` 的示例：

[复制代码](#)

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10
11 fn doit(x: impl TraitA) {}
12 // 等价于
13 // fn doit<T: TraitA>(x: T) {}
14
15 fn main() {
16     let a = Atype;
17     doit(a);
18     let b = Btype;
19     doit(b);
```



```
20     let c = CType;  
21     doit(c);  
22 }
```

dyn trait 的示例：

[复制代码](#)

```
1 struct Atype;  
2 struct Btype;  
3 struct CType;  
4  
5 trait TraitA {}  
6  
7 impl TraitA for Atype {}  
8 impl TraitA for Btype {}  
9 impl TraitA for CType {}  
10  
11 fn doit(x: &dyn TraitA) {} // 注意这里用了引用形式 &dyn TraitA  
12  
13 fn main() {  
14     let a = Atype;  
15     doit(&a);  
16     let b = Btype;  
17     doit(&b);  
18     let c = CType;  
19     doit(&c);  
20 }
```

两种都可以。那么它们的区别是什么呢？

impl trait 用的是编译器静态展开，也就是编译时具化（单态化）。上面那个 impl trait 示例展开后类似于下面这个样子。

[复制代码](#)

```
1 struct Atype;  
2 struct Btype;  
3 struct CType;  
4  
5 trait TraitA {}  
6  
7 impl TraitA for Atype {}
```

```
8  impl TraitA for Btype {}
9  impl TraitA for Ctype {}
10
11 fn doit_a(x: Atype) {}
12 fn doit_b(x: Btype) {}
13 fn doit_c(x: Ctype) {}
14
15 fn main() {
16     let a = Atype;
17     doit_a(a);
18     let b = Btype;
19     doit_b(b);
20     let c = Ctype;
21     doit_c(c);
22 }
```

而 dyn trait 的版本不会在编译期间做任何展开，dyn TraitA 自己就是一个类型，这个类型相当于一个代理类型，用于在运行时代理相关类型及调用对应方法。既然是代理，也就是调用方法的时候需要多跳转一次，从性能上来说，当然要比在编译期直接展开一步到位调用对应函数要慢一点。

静态展开也有问题，就是会使编译出来的内容体积增大，而 dyn trait 就不会。所以它们各有利弊，可以根据需求视情况选择。另外，**impl trait 和 dyn trait 都是消除类型参数的办法。**

那它们和 enum 相比呢？

enum 是封闭类型集，可以把没有任何关系的任意类型包裹成一个统一的单一类型。后续的任何变动，都需要改这个统一类型，以及基于这个 enum 的模式匹配等相关代码。而 impl trait 和 dyn trait 是开放类型集。只要对新的类型实现 trait，就可以传入使用了 impl trait 或 dyn trait 的函数，其函数签名不用变。

上述区别对于库的提供者非常重要。如果你提供了一个库，里面的多类型使用的 enum 包装，那么库的使用者没办法对你的 enum 进行扩展。因为一般来说，我们不鼓励去修改库里面的代码。而用 impl trait 或 dyn trait 就可以让接口具有可扩展性。用户只需要给他们的类型实现你的库提供的 trait，就可以代入库的接口使用了。

因为 Vec 中要求每一个元素是同一种类型，不能将不同的类型实例放入同一个 Vec。而利用 trait object，我们可以“绕”过这个限制。

请看示例：

 复制代码

```
1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10
11 fn main() {
12     let a = Atype;
13     let b = Btype;
14     let c = Ctype;
15     let v: Vec<&dyn TraitA> = vec! [&a, &b, &c];
16 }
```

成功了，不同类型的实例（实际是实例的引用）竟然被放进了同一个 Vec 中，强大！你可以自己尝试一下，将不同类型的实例放入 HashMap 中。

既然 trait object 这么好用，那是不是可以随便使用呢？不是的。除了前面提到的性能损失之外，还有一个问题，不是所有的 trait 都可以做 dyn 化，也就是说，不是所有的 trait 都能转成 trait object 使用。

哪些 trait 能用作 trait object?

只有满足对象安全（object safety）的 trait 才能被用作 trait object。Rust 参考手册上有关于 [object safety](#) 的详细规则，比较复杂。这里我们了解常用的模式就行。

安全的 trait object：

 复制代码

```
1 trait TraitA {  
2     fn foo(&self) {}  
3     fn foo_mut(&mut self) {}  
4     fn foo_box(self: Box<Self>) {}  
5 }
```

不安全的 trait object:

 复制代码

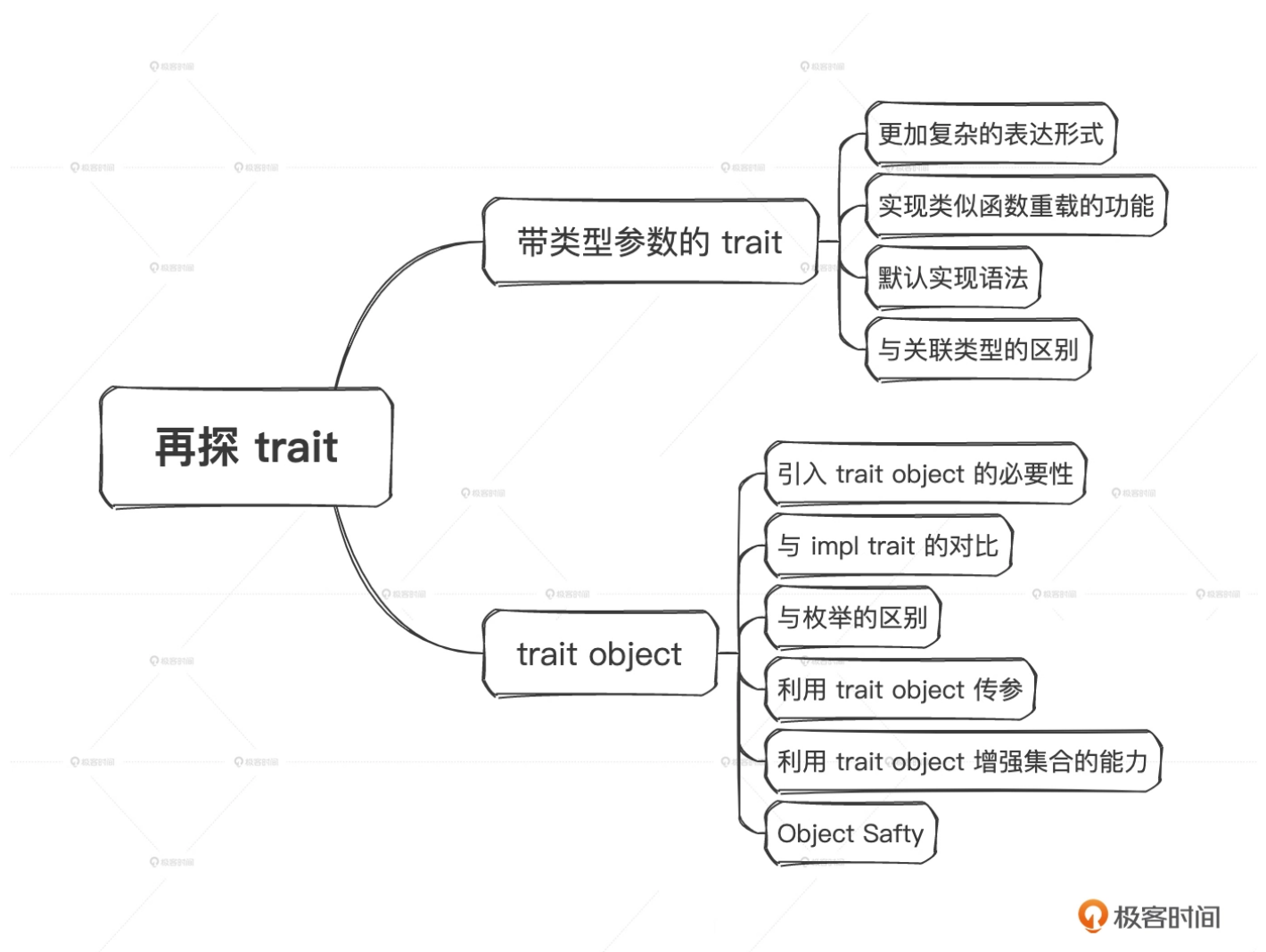
```
1 trait NotObjectSafe {  
2     const CONST: i32 = 1; // 不能包含关联常量  
3  
4     fn foo() {} // 不能包含这样的关联函数  
5     fn selfin(self); // 不能将Self所有权传入  
6     fn returns(&self) -> Self; // 不能返回Self  
7     fn typed<T>(&self, x: T) {} // 方法中不能有类型参数  
8 }
```

规则确实比较复杂，你可以简单记住几种场景。

1. 不要在 trait 里面定义构造函数，比如 new 这种返回 Self 的关联函数。你可以发现，确实在整个 Rust 生态中都没有将构造函数定义在 trait 中的习惯。
2. trait 里面尽量定义传引用 &self 或 &mut self 的方法，而不要定义传值 self 的方法。

并不是所有的 trait 都能以 trait object 形式 (dyn trait) 使用，实际上，以 dyn trait 使用的场景可能是少数。所以你可以在遇到编译器报错的时候再回头来审视 trait 定义得是否合理。大部分情况下可以放心使用。

小结



在这节课的前半部分，我们讲解了 trait 中带类型参数的情况。各种符号组合起来，确实越来越复杂了。不过还是那句话，模式就那几种，只要花点时间熟悉理解，其实并不难。开始的时候能认识就行，后面在实践中再慢慢掌握。

我们使用带类型参数的 trait 实现了其他语言中函数重载的功能。看起来途径有点曲折，但是带给了我们一条全新的思路：以往的语言必须给自身添加各种特性来满足用户的要求，在 Rust 中，用好 trait 就能搞定。这让我们对 Rust 的未来充满期待，随着时间的发展，它不会像 C++、Java 那样永不停歇地添加可能会导致组合爆炸的新特性，而让自身越来越臃肿。

我们还讨论了带类型参数的 trait 与关联类型的区别。它们之间并不存在绝对优势的一方，在合适的场景下选择合适的方案是最重要的。

然后通过一个问题：如何让一个 Rust 函数返回可能的多种类型？推导出了引入 trait object 方案的必要性。整个推导过程比较曲折，同时也是对 Rust 类型系统的一次精彩探索。

在这个探索过程中，我们和 Rustc 小助手成为了好朋友，在它的协助下，我们找到了最佳方案。

最后我们了解了 trait object 的一些用途，并讨论了 trait object、impl trait，还有使用枚举对类型进行聚合这三种方式之间的区别。类型系统（类型 + trait）是 Rust 的大脑，你可以多加练习，熟悉它的形式，掌握它的用法。

思考题

请谈谈在函数参数中传入 `&dyn TraitA` 与 `Box<dyn TraitA>` 两种类型的区别。

欢迎你把思考后的结果分享到评论区，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (14)

最新 精选



哄哄

2023-11-10 来自北京

rust生命周期的独特设计，导致了该语言需要设计一些处理方式应对特殊情况，比如生命周期的标注（主要是给编译器进行代码处理时的提示）。事实上，我们在日常开发中应该避免一些陷入复杂情况的方式：比如，传入参数都用引用（borrow），传出结果都应该是owner。rust 也为我们提供了处理各种情况的工具。所以，一般来说，我们应该在传入参数的时候用`&dyn T`，传出结果用`Box<dyn T>`。

作者回复：真不错 👍



👍 9



鸠摩智

2023-11-10 来自江苏

`&dyn TraitA`没有所有权，而`Box<dyn TraitA>`有所有权。

作者回复: 对的



👍 3



-

2023-11-10 来自北京

最后安全的trait object听的似懂非懂，为什么是安全的，为什么是不安全的？希望具体讲下原因

作者回复: 这里面的设计原理确实比较dirty了。我先给出三篇文章参考，你可以深入下去研究：<http://rustcc.cn/article?id=e80a98d7-4adb-4d39-bf0a-4e079bcb1835> <https://segmentfault.com/a/1190000022104780> <https://rust-lang.github.io/rfcs/0255-object-safety.html>

共 4 条评论 >

👍 2



哄哄

2023-11-10 来自北京

关联类型之所以要单独设计，因为编译器可以通过输入判断具体类型，而无法推断出输出类型，所以，输出的类型需要明确指定

作者回复: 思考方式很棒！



👍 2



superggn

2023-12-15 来自北京

思考题：&dyn traitA 和 Box<dyn traitA> 严格来说都是指向 actual object 的指针，只不过`&`叫做引用，`Box`叫做智能指针，这俩都是固定大小的，所以都能用在 trait object

区别：

&dyn traitA 是一个不拿所有权的指针 => 所以经常用在参数里

Box<dyn traitA> 是拥有内部数据所有权的指针 => 所以经常用在返回值里，参数里应该也能用

作者回复：对的。👍



👍 1



小可爱(`へ`*)ノ

2023-12-28 来自四川

基础部分非常详细，建议老师可以深入讲一下trait object的内存相关知识，以及trait object会丢失本身类型信息这些注意点。

作者回复：好滴，记下了，后面会持续输出。



superggn

2023-12-15 来自北京

笔记：

trait 可以带泛型 generics

trait 关联类型 associated type 跟泛型类似，但必须在 impl 的时候就具化

泛型就随便啥时候具化都行了

``fn doit(i: u32) -> impl TraitA` =>` 某一种实现了 TraitA 的类型

``fn doit(i: u32) -> Box<dyn TraitA>` =>` 多种实现了 TraitA 的类型

前两次刷 rbe 和 rustbook 的时候都没反应过来为啥叫 trait object, 看这课终于明白了，原来 trait object 指的是 typeOuter 的 OuterSizedObj, 然后这个 OuterSizedObj 里面可以包裹各种

实现了 traitA 的 type 对应 obj

还是得多刷点儿资料，多看对于同一概念的不同表达，明白的快

作者回复：真棒！



-Hedon 🌈

2023-11-21 来自湖北

思考题：&dyn TraitA 是借用，Box<dyn TraitA> 会转移所有权。

通过下面的程序可以测试出来：

```
fn doit3(t1: &dyn TraitA, t2: Box<dyn TraitA>) {  
    println!("{:?}", t1);  
    println!("{:?}", t2)  
}
```

```
fn main() {  
    let a = AType;  
    let b = BType;  
    doit3(&a, Box::new(b));  
    println!("{:?}", a);  
    println!("{:?}", b);  
}
```

输出：

```
error[E0382]: borrow of moved value: `b`  
--> examples/trait_object.rs:29:22  
   |  
26 |     let b = BType;  
   |         - move occurs because `b` has type `BType`, which does not implement the `Copy` trait
```

```

27 |     doit3(&a, Box::new(b));
    |           - value moved here
28 |     println!("{:?}", a);
29 |     println!("{:?}", b);
    |           ^ value borrowed here after move

```

作者回复: 👍 👍



老大

2023-11-21 来自河南

不要在 trait 里面定义构造函数，比如 new 这种返回 Self 的关联函数。你可以发现，确实在整个 Rust 生态中都没有将构造函数定义在 trait 中的习惯。

这句话，在上面的例子中 确实有在trait中定义了new函数返回self的。怎么就感觉有冲突呢？

作者回复: 观察得很细致，上面的例子主要是为了展现可能出现的写法。后面的说明加入了object safety的因素，以及社区的约定，所以看起来好像不一致，实际是一致的。



Ransang

2023-11-18 来自上海

```

struct Atype;
struct Btype;
struct Ctype;

```

```

trait TraitA {}

```

```

impl TraitA for Atype {}
impl TraitA for Btype {}
impl TraitA for Ctype {}

```

```

fn doit(i: u32) -> &'static dyn TraitA { // 注意这里的返回类型换成了 dyn TraitA
    if i == 0 {
        return &Atype
    } else if i == 1 {
        return &Btype
    }
}

```

```
} else {  
    return &Ctype  
}  
}
```

老师 我这种也能通过编译，我一开始尝试用`&dyn TraitA`没通过编译，小助手报错`error[E0106]: missing lifetime specifier`并提示我用`&'static dyn TraitA`，在字符串那节您提过 `'static` 表示这个引用可以贯穿整个程序的生命周期，想问下您这段代码为什么加了`'static`就可以通过编译，以及这个生命周期的概念又是怎么回事，谢谢了

作者回复：👍，这种探索的方式很棒。`'static` 确实起这个作用。在 Rust 中，`'static` 生命期是整个程序的持续时间。当你在类型前面加上 `'static` 时，你告诉 Rust 编译器这个引用应该在整个程序运行期间都有效。

但是这并不意味着你可以返回一个指向局部变量的 `'static` 引用。这是因为局部变量在函数返回时就会被销毁，所以任何指向它们的引用在函数返回后都将变得无效。这个 `'static` 告诉 Rust 编译器这个引用应该在整個程序运行期间都有效，于是Rust就信了，即使实际上这是不可能的，所以还是要用`Box`。从这一点也能看出Rust编译器也不完美。

然后第20讲会讲生命期的概念，但是也不会特别深入。初学Rust的时候，先不用花大量时间在这上面，可以把这个问题留到后面去。

